

Submission - Exercise 2

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Group 4
 - Evan Martino
 - Marco Birchler
 - Roman Babukh

Repository

- GitHub repository: https://github.com/chechmek/EDPO_FS26_Project

Team contribution

- Roman Babukh focused primarily on the implementation of the project this week.
- Marco Birchler worked on the documentation and submission.
- Evan Martino supported the other team members in both tasks.

Project Abstract

The project demonstrates an event-driven social media simulation using the **Event Notification** EDA pattern, Python 3.12, Kafka, and PostgreSQL with three services:

- **post-service**: stores posts and emits **PostCreated**
- **interaction-service**: stores likes/comments/follows and emits interaction events
- **notification-service**: consumes interaction/post events and **builds notifications in its own datastore**

EDA Pattern: Event Notification

Definition:

In the Event Notification pattern, a service emits events to inform other services that something has happened in its domain. The emitting service does not know which services (if any) will consume the event, and does not depend on them. Consumers subscribe independently and react to the events they care about.

Key characteristics:

- The producer has no knowledge of consumers
- Events carry only enough information to notify — consumers may need to query back for full state if required
- Communication is asynchronous and one-directional

Why we chose this pattern:

The interaction domain (likes, comments, follows) naturally produces side effects that other parts of the system care about — in particular, notifying post owners. Using Event Notification means **interaction-**

`service` is not coupled to `notification-service`: it simply records facts about user interactions and broadcasts them. This keeps each service focused on its own bounded context and allows the notification logic to evolve independently.

Mapping to our implementation:

- `interaction-service` is the **producer**: it emits `PostLiked`, `PostUnliked`, `CommentAdded`, `UserFollowed` events to Kafka after persisting domain state.
- `notification-service` is the **consumer**: it listens to `social.interaction.events` and `social.post.events` and decides independently which events warrant a user notification.
- There is **no direct synchronous call** from `interaction-service` to `notification-service`. Dependency flows only from consumer to broker, never between services directly.

Architecture

Event Flow

1. Client calls `post-service` or `interaction-service` HTTP APIs.
2. `post-service` persists the post and publishes `PostCreated` to `social.post.events`.
3. `interaction-service` persists the interaction and publishes the corresponding event to `social.interaction.events`.
4. `notification-service` consumes from both topics. It maintains a local `post_owners` table (populated from `PostCreated`) so it can resolve the correct recipient for interaction notifications.
5. For relevant events (`PostLiked`, `CommentAdded`), `notification-service` writes a notification record and exposes it through its own HTTP API.



What Was Implemented for Exercise 2

- Implemented a Kafka-based, decoupled Event Notification flow between interaction and notification domains.
- Implemented publisher logic in `interaction-service` for interaction events (`PostLiked`, `PostUnliked`, `CommentAdded`, `UserFollowed`).
- Implemented subscriber logic in `notification-service` consuming interaction and post events.

- Implemented notification creation for relevant events (`PostLiked`, `CommentAdded`) and non-notifying handling for others.
- Implemented independent data ownership per service (no cross-service DB writes).

How to Verify the Exercise Deliverable

1. Start the system:
 - `make venv`
 - `make install`
 - `make up`
2. Generate activity:
 - `make demo`
 - or run `scripts/generate_load.py`
3. Observe event flow:
 - Open Kafka UI at `http://localhost:8080`
 - Inspect `social.post.events` and `social.interaction.events`
4. Verify notification behavior:
 - Query `notification-service` endpoint `GET /users/{user_id}/notifications`
 - Confirm notifications are created for `PostLiked` and `CommentAdded`

Scope and Limitations

- Current notification generation is intentionally limited to owner-facing `PostLiked` and `CommentAdded`.
- `PostUnliked` and `UserFollowed` are consumed but do not create notification records.
- This submission focuses on demonstrating Event Notification and service decoupling for Exercise 2.

Technical Reference

Detailed technical repository documentation (event schemas, data models, API details, setup/runbook) is provided in:

- `README.md`